TQ Level Description Language

© 1992-2003 TQworld, LLC

The following tables define the level description commands used in tranquility.
There's no guarantee that any versions of tranquility support any or all of these commands,
or that the syntax described here is accurate or complete. But as of May 2003, it's pretty close.

Good Luck! - David Cook & Bill Romanowski

If you've got questions, email: tranquility@mac.com

WORLD LEVEL COMMANDS

I - Initialize a room

Syntax:	**I**

The **I** command is used to Initialize the client-side executable. It forces the client-side to reset all major variables and arrays and prepare for a new room to be downloaded. As such, the **I** command should be the FIRST DRAWING command received by the system. Each new *room* must have a **I** command at the top or it will not load properly.

M - Assign a message to a message queue

Syntax:	**M** *n string*

The **M** command assigns a *string* to one of the message queues. These can be used to hold text or URL's or other string information. There are 256 message queues so the value of *n specifies which queue as a number between **0** and **255**. Note that each string can not be longer than 255 characters.*

*Queue elements with n set to **128** to **255** are persistant between levels, whereas queue elements with n set to **0** to **127** are automatically cleared when a **I** world command is encountered. Any persistant message may be overwritten at any time and may be cleared from memory by setting string to a single period (.).*

Note that the underscore character (_) is replaced dynamically with a space... allowing spaces to be placed in messages at the start or end of the message.

! - Speak a line of text

Syntax:	**!** *string*

The **!** command allows you to transmit a string of text that will be spoken by voice-translation software on the users computer. You should not overuse this command and you should not put critical information in any string since the user may not have voice-translation software at their end, or may have it turned off.

W - Define a rooms world space

Syntax:	**W** *level switchbits add1 add2 add3 add4 gravity upclip damp xpos ypos zpos apos mode mousex mousey level*

The **W** command defines the rooms world space. This command **should** appear **immediately after** the **I** command to allow the client-side to setup world space for the room being delivered.

There should only be one **W** command in a level. Subsequent **W** commands will overwrite the previous one but the *xpos*, *ypos*, *zpos* and *apos* parameters set the user's location immediately so you may see a glitch if you use more than one **W** command.

The current **W** command has the following parameters:

Parameter	Format	Range	Definition
level	*float*	**any**	This sets the initial value of the current level. The designer may create rooms of any number of levels - this simply sets the starting point.
switchbits	*long*	**any**	This sets the initial value for the switchbits. The **E**vent command allows this parameter to be modified based on events that occur in the system, thus this value most likely changes throughout the users run. The 'current' value is sent back to the server when instructed as one of info values - and reflects the status of the user (e.g., what events they triggered) at that point in time.
add1	*float*	**any**	A value that is added to the current level when instructed to do so via **E**vent.
add2	*float*	**any**	A value that is added to the current level when instructed to do so via **E**vent.
add3	*float*	**any**	A value that is added to the current level when instructed to do so via **E**vent.
add4	*float*	**any**	A value that is added to the current level when instructed to do so via **E**vent.
gravity	*float*	**any**	The desired gravity for this room. A value of zero turns gravity off.
upclip	*float*	**any**	This limits the maximum up-direction (Y) speed. A value of around 20.0 is good in situations where you may wish to avoid flying off through power-stack runs.

damp	*float*	**any**	This sets the viscosity of the motion dynamics. A value of **.006** is most pleasing.
xpos	*float*	**any**	This sets the users initial X position when a level first loads. Note that a value of **0** will cause this axis to retain it's previous position.
ypos	*float*	**any**	This sets the users initial Y position when a level first loads. Note that a value of **0** will cause this axis to retain it's previous position.
zpos	*float*	**any**	This sets the users initial Z position when a level first loads. Note that a value of **0** will cause this axis to retain it's previous position.
apos	*float*	**any**	This sets the users turn angle when a level first loads. Values that make sense should remain in the **0** to **360** value range.
mode	*float*	**any**	This value is currently ignored.
mousex	*float*	**any**	This sets the mouse sensitivity in the X direction. Use **1.0** as a default value here.
mousey	*float*	**any**	This sets the mouse sensitivity in the Y direction. Use **1.0** as a default value here.
level	*string*	**any**	This is reserved for the server software to transmit a string to the browser. The browser aways echos this back in **P16**.

A - Define an Aggregate Group

Syntax: **A** *number*

The **A** command starts a **GROUP** definition. Only **GROUP** commands should appear **after** the **A** command until another **A** command appears. The *number* indicates which group this is, and should be sequentially numbered starting at **0** with the largest possible group being **255**.

When a **A** command is encountered the TQ client puts all subsquent commands, up to the next **A** command into a bin associated with the assigned group. Note that within the server-side code the **A** command must be defined as a *focus* type within the **tq.lang** file so that focus can be properly maintained by the room-builder.

GROUP LEVEL COMMANDS
The following commands MUST appear under an **A** Aggregate Group world command.

E - Event Handlers

Syntax: **E** *notebits sysbits orbits xorbits*

The **E** command allows you to specify how events are handled. You can define 32 **E** records per aggregate group. The event parameters are:

Parameter	Format	Range	Definition
notebits	*string*	**0's and 1's**	This parameter consists of a string of **0**'s and/or **1**'s. When this event is triggered, **N** (note) events are called in MSB (most significant bit) order.
sysbits	*string*	**0's and 1's**	This parameter consists of a string of **0**'s and/or **1**'s. When this event is triggeredd each sysbit which is a **1** triggers a system event. *sysbits* are called in MSB (most significant bit) order with a total of 8 sysbits (01234567). The meaning of the sysbits are:
orbits	*long*	**any**	On events the *orbits* are **OR**ed with *switchbits* (switchbits is defined in the **W** command). Note that *orbits* occurs first and *xorbits* occur second.
xorbits	*long*	**any**	On events the *xorbits* are **XOR**ed with *switchbits* (switchbits is defined in the **W** command). Note that *orbits* occurs first and *xorbits* occur second.

Bit	Description
0	G_level += add1 (from W command)
1	G_level += add2
2	G_level += add3
3	G_level += add4
4	TQ contacts the host cgi-bin with e0
5	TQ contacts the host cgi-bin with e1
6	TQ contacts the host cgi-bin with e2
7	TQ contacts the host cgi-bin with e3

T - Define a Timer

Syntax: **T** *eventbits ticks offset noise delay lstart lend times restart mode*

The **T** command allows you to define timers which are used to generate asynchronous events that are independent of user interaction with the environment. The primary intent is to generate background music but timers can trigger anything the event system is attached to.

The timers are associated with whatever aggregrate group they're defined within... thus they are aggregate specific commands.

There are 256 timers available. Timers greater than 256 are ignored in the data stream. There's no dependency in the TQ engine that would prohibit expanding beyond 256 if needed, but they get scanned -every frame- so each **T** declaration directly affects the frame rate to some small degree and they can add up.

Timers use a combination of timing from both the frame rate **and** the 1/60 second system clock. It's a bit weird but that's what gives it it's eno-esque charm. It does it's best to be accurate but some timers may be missed and the time base will slide a bit depending on system load, GL driver, and phase of the moon. Expect a deviation of between 2% to 10% of clock slop in your timer events.

To make the best use of timers avoid making levels that depend on a single timer-triggered event to make something happen, it might not do what you want, when you want it. Also keep in mind that triggering 100 sounds at the same time is obviously not recommended. TQ will do it, but it won't be pretty. Use *offset* to stagger execution of timers that have *ticks* of even multiples. For example, if you have 3 timers with *ticks* of 60, 120 and 240... than every 4 seconds all 3 timers will be triggered. 3 of these is fine but dozens of them will start killing the frame rate.

The timer parameters are:

Parameter	Format	Range	Definition
eventbits	*string*	**0's and 1's**	This field is up to 32 characters long where each character is either a **0** or **1**. The corresponding bit (MSB first) equates to events and specifies **0** a non-trigger or **1** a trigger of that event.
ticks	*long*	**any**	This parameter sets the frequency of the timer in 1/60 seconds increment.
offset	*long*	**any**	This is the phase of the timer and is added to the system clock during the *ticks* compare. Use this parameter to add off-beats or syncapation to your timers
noise	*long*	**any**	This is a random amount of dither that should be added to *ticks* before each compare. Set this parameter to 0 if not used, as it is very expensive. small values are best (1-10). Note that even though this value is a positive value TQ will treat it as a +/- peak thus the value **1** will causes noise to swing +/-**1**, etc.
delay	*long*	**any**	This parameter specifies a number of frames that MUST pass before this timer is triggered. This gives you a method of defering a timer until a certain number of frames have passed.
lstart	*float*	**any**	This parameter sets the level (game level) at which this particular timer event becomes activated. If the current play level is less than *lstart* than the timer does not activate. Once the current play level has reached *lstart* than the timer may activate depending on the settings of the other parameters.
lend	*float*	**any**	This parameter sets the level (game level) above which this particular timer event becomes deactivated. If the current play level is above the value specified by *lend* than the timer does not activate.
times	*long*	**any**	This parameter sets the maximum number of times that a timer will trigger. If this value is set to **0** than no limit to the number of times a timer can trigger will be set (except as goverened by the other parameters).
restart	*long*	**0** or **1**	If this value is a one **1** than when the timre times out (*times* non-zero) it is automatically reset, including any delay.
mode	*long*	**zero**	This parameter is currently reserved but must be set to zero for the current version.

B - Define a Bouy

Syntax: **B** *eventsin eventsout rin noise rout xpos ypos zpos lstart lend mode*

The **B** command defines a *Bouy. Buoys act like timers (please read the **T** command first) in that they trigger events, however Buoys different in that they use the user's proximity instead of system ticks, to trigger the event. Buoys can be used to tell if the user enters or exits a spherical region in TQ space.*

*As with **T**imers, there can be a maximum of 256 **B** definitions. The current aggregate at definition time is used to point at the correct Bouy at runtime. Note that rin and rout parameters are used to determine the radius of the buoy bubble. This is to provide a dead zone so buoy events aren't triggered continously if the user is sitting still.*

When a level first initializes, the TQ software automatically figures out which bubble the user is in or out of and if the user appears "inside" the bouy isn't triggered until the user leaves the bubble (so a bunch of events don't go off on initialization).

Since bubbles don't move from their initial position you can not use Bouys for collision detection with moving geometry.

Parameter	Format	Range	Definition
eventsin	*string*	**0's and 1's**	This field is up to 32 characters long where each character is either a **0** or **1**. The corresponding bit (MSB first) equates to events and specifies **0** a non-trigger or **1** a trigger of that event **when a bouy is entered**.
eventsout	*string*	**0's and 1's**	This field is up to 32 characters long where each character is either a **0** or **1**. The corresponding bit (MSB first) equates to events and specifies **0** a non-trigger or **1** a trigger of that event **when a bouy is exited**.
rin	*float*	**any**	This is the inside radius of the bouy bubble. This should be less than *rout*.
rout	*float*	**any**	This is the outside radius of the bouy bubble. This should be greater than *rin*.
xpos	*float*	**any**	This is the X position of the center of the bubble.
ypos	*float*	**any**	This is the Y position of the center of the bubble.
zpos	*float*	**any**	This is the Z position of the center of the bubble.
lstart	*float*	**any**	This parameter sets the level (game level) at which this particular buoy event becomes activated. If the current play level is less than *lstart* than the buoy does not activate. Once the current play level has reached *lstart* than the buoy may activate depending on the settings of the other parameters.
lend	*float*	**any**	This parameter sets the level (game level) above which this particular buoy event becomes deactivated. If the current play level is above the value specified by *lend* than the buoy does not activate.
mode	*long*	**zero**	This parameter is currently reserved but must be set to zero for the current version.

C - Define a color space

Syntax: **C 0** *hue sat val alpha inc*

This form of the **C** command will define a single color. The color type is **0** (zero). The color parameters are:

Parameter	Format	Range	Definition
hue	*float*	**0.0 - 1.0**	This sets the hue of the color.
sat	*float*	**0.0 - 1.0**	This sets the saturation of the color.
val	*float*	**0.0 - 1.0**	This sets the value (intensity) of the color.
alpha	*float*	**0.0 - 1.0**	This sets the alpha (transparency) of the color. Colors should have an alpha of between **.7** and **1.0** for 'normal' platforms - though any value between **0** and **1** is allowed.
inc	*float*	**any**	This sets an increment to add to *hue* for each frame. Since this is added on a per-frame basis very small numbers work the best.

Syntax: **C 1** *hue sat val alpha CYf CYp XYZamp Mf Ma*

This form of the **C** command will create an AUTO HUE SIN. The color type is **1** (zero). The color parameters are:

Parameter	Format	Range	Definition
hue	*float*	**0.0 - 1.0**	This sets the hue of the color.
sat	*float*	**0.0 - 1.0**	This sets the saturation of the color.
val	*float*	**0.0 - 1.0**	This sets the value (intensity) of the color.
alpha	*float*	**0.0 - 1.0**	This sets the alpha (transparency) of the color. Colors should have an alpha of between **.7** and **1.0** for 'normal' platforms - though any value between **0** and **1** is allowed.
CYf	*float*	**any**	This value can be thought of as specifying the speed of the hue change. If this value is zero hue change is muted. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) to the hue.
CYp	*float*	**any**	This value specifies the phase, or offset, of each hue generated. This can be used to shift the hue's.
XYZamp	*float*	**any**	This value specifies how much the position of where the color is drawn affects the color.

Mf	*float*	**any**	This value specifies the master hue frequency.
Ma	*float*	**any**	This value specifies the master hue amplitude.

Note that the TQ function **$?RGB2HSV** allows you an easy way to convert RGB values into HSV values compatible with this routine.

Syntax: **C 2** *hue sat val alpha CYf CYp XYZamp Mf Ma*

This form of the **C** command will create an AUTO HUE COS. The color type is **2** (zero). The color parameters are:

Parameter	Format	Range	Definition
hue	*float*	**0.0 - 1.0**	This sets the hue of the color.
sat	*float*	**0.0 - 1.0**	This sets the saturation of the color.
val	*float*	**0.0 - 1.0**	This sets the value (intensity) of the color.
alpha	*float*	**0.0 - 1.0**	This sets the alpha (transparency) of the color. Colors should have an alpha of between **.7** and **1.0** for 'normal' platforms - though any value between **0** and **1** is allowed.
CYf	*float*	**any**	This value can be thought of as specifying the speed of the hue change. If this value is zero hue change is muted. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) to the hue.
CYp	*float*	**any**	This value specifies the phase, or offset, of each hue generated. This can be used to shift the hue's.
XYZamp	*float*	**any**	This value specifies how much the position of where the color is drawn affects the color.
Mf	*float*	**any**	This value specifies the master hue frequency.
Ma	*float*	**any**	This value specifies the master hue amplitude.

Note that the TQ function **$?RGB2HSV** allows you an easy way to convert RGB values into HSV values compatible with this routine.

Syntax: **C 10** *h1 s1 v1 a1 i1 h2 s2 v2 a2 i2*

This form of the **C** command will create a GRADIENT COLOR. This sets a gradient spread of colors through the geometry it is assigned to. The color parameters are:

Parameter	Format	Range	Definition
h1	*float*	**0.0 - 1.0**	This sets the hue of the starting color. Because hue wraps around in color space you can set to be large values to generate multiple rainbow gradients (e.g., a value of 2.0 will cycle through the hues twice, depending on the number of geometries).
s1	*float*	**0.0 - 1.0**	This sets the saturation of the starting color.
v1	*float*	**0.0 - 1.0**	This sets the value (intensity) of the starting color.
a1	*float*	**0.0 - 1.0**	This sets the alpha (transparency) of the starting color. Colors should have an alpha of between **.7** and **1.0** for 'normal' platforms - though any value between **0** and **1** is allowed.
i1	*float*	**any**	This is the increment that is applied to step through over time. A value of **0** does not cause the gradient to change. Higher values cause the gradient to cycle over time. Keep this value small, in the order of **.001** as it is executed for each geometry that uses it.
h2	*float*	**0.0 - 1.0**	This sets the hue of the ending color. Because hue wraps around in color space you can set to be large values to generate multiple rainbow gradients (e.g., a value of 2.0 will cycle through the hues twice, depending on the number of geometries).
s2	*float*	**0.0 - 1.0**	This sets the saturation of the ending color.
v2	*float*	**0.0 - 1.0**	This sets the value (intensity) of the ending color.
a2	*float*	**0.0 - 1.0**	This sets the alpha (transparency) of the ending color. Colors should have an alpha of between **.7** and **1.0** for 'normal' platforms - though any value between **0** and **1** is allowed.
i2	*float*	**any**	This is the increment that is applied to step through over time. A value of **0** does not cause the gradient to change. Higher values cause the gradient to cycle over time.

			Keep this value small, in the order of **.001** as it is executed for each geometry that uses it.

Note that the TQ function **$?RGB2HSV** allows you an easy way to convert RGB values into HSV values compatible with this routine.

G - Define geometry

Syntax: **G 1** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Rectangle**. The geometry type is **1** (one). The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the rectangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the rectangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the rectangle will be a square.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This is which defined **C** color command to assign to the fourth corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 2** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Triangle**. The geometry type is **2**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the triangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the triangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the triangle will be equalateral.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This value is ignored

Syntax: **G 3** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Diamond**. The geometry type is **3**). The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the diamond. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the diamond. Good values would be in the 15 to 30 range. If *sy* matches *sx* the diamond will be a rotated square.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This is which defined **C** color command to assign to the fourth corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 4** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Alternate Triangle**. This triangle differs from a type **2** triangle in that it is rotated 90 degress from the type **2**. The geometry type is **4**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the triangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the triangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the triangle will be equalateral.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This value is ignored

Syntax: **G 5** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Alternate Triangle**. This triangle differs from a type **4** triangle in that it is rotated 90 degress from the type **4**. The geometry type is **5**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the triangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the triangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the triangle will be equalateral.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.

zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This value is ignored

Syntax: **G 6** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Alternate Triangle**. This triangle differs from a type **5** triangle in that it is rotated 90 degress from the type **5**. The geometry type is **6**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the triangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the triangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the triangle will be equalateral.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This value is ignored

Syntax: **G 7** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Alternate Triangle**. This triangle differs from a type **6** triangle in that it is rotated 90 degress from the type **6**. The geometry type is **7**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the triangle. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the triangle. Good values would be in the 15 to 30 range. If *sy* matches *sx* the triangle will be equalateral.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second corner. The value **0**

			(zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third corner. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This value is ignored

Syntax: **G 16** *size message xo yo zo xr yr zr c1*

This Geometry command defines a **String Object**. This allows a string of text to be drawn as a platform. The geometry type is **16**. The geometry parameters are:

Parameter	Format	Range	Definition
size	*float*	**any**	This is the desired size of the text.
message	*long*	**0 - 255**	This is the number of the message queue to draw (see the **M** command).
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the string. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 17** *size message xo yo zo xr yr zr c1*

This Geometry command defines a **Letter Object**. This takes a string of text and draws each letter as the next platform. The geometry type is **17**. The geometry parameters are:

Parameter	Format	Range	Definition
size	*float*	**any**	This is the desired size of the text.
message	*long*	**0 - 255**	This is the number of the message queue to draw (see the **M** command).
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the string. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 20** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Solid Sphere**. The geometry type is **20**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the sphere. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the sphere. Good values would be in the 15 to 30 range. If *sy* matches *sx* the sphere will be round.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the sphere. The entire sphere becomes this color. The value **0** (zero) means the *first color defined in this Aggregate*

			Group and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 21** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Solid Cube**. The geometry type is **21**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the cube. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the cube. Good values would be in the 15 to 30 range. If *sy* matches *sx* the cube will be symetrical in all axis.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the cube. The entire cube becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 22** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Solid Pyrimid**. The geometry type is **22**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the pyrimid. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the pyrimid. Good values would be in the 15 to 30 range. If *sy* matches *sx* the pyrimid will be equalateral in it's vertical cross-section.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the pyrimid. The entire pyrimid becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 23** *thickness radius xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Solid Donut**. The geometry type is **23**. The geometry parameters are:

Parameter	Format	Range	Definition
thickness	*float*	**any**	This is the thickness of the donut's ring.
radius	*float*	**any**	This is the radius of the donut.

xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the donut. The entire donut becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 24** *width radius xo yo zo xr yr zr c1 c2 c3 c4 c5 c6*

This Geometry command defines a **Atomic Shell**. The geometry type is **24**. The geometry parameters are:

Parameter	Format	Range	Definition
width	*float*	**any**	This is the width of each ring. Good values would be in the .05 to .5 range.
radius	*float*	**any**	This is the radius the ring. Good values would be in the 15 to 30 range.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This is which defined **C** color command to assign to the forth ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c5	*index*	**index**	This is which defined **C** color command to assign to the fifth ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c6	*index*	**index**	This is which defined **C** color command to assign to the sixth ring of the atomic shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 25** *width radius xo yo zo xr yr zr c1 c2 c3 c4 c5 c6*

This Geometry command defines a **Wire Shell**. The geometry type is **25**. The geometry parameters are:

Parameter	Format	Range	Definition
width	*float*	**any**	This is the width of each ring. Good values would be in the .05 to .5 range.
radius	*float*	**any**	This is the radius the ring. Good values would be in the 15 to 30 range.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the first ring of the wire shell. The

			value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This is which defined **C** color command to assign to the second ring of the wire shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c3	*index*	**index**	This is which defined **C** color command to assign to the third ring of the wire shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c4	*index*	**index**	This is which defined **C** color command to assign to the forth ring of the wire shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c5	*index*	**index**	This is which defined **C** color command to assign to the fifth ring of the wire shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c6	*index*	**index**	This is which defined **C** color command to assign to the sixth ring of the wire shell. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...

Syntax: **G 30** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Wire Sphere**. The geometry type is **30**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the sphere. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the sphere. Good values would be in the 15 to 30 range. If *sy* matches *sx* the sphere will be round.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the sphere. The entire sphere becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 31** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Wire Cube**. The geometry type is **31**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the cube. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the cube. Good values would be in the 15 to 30 range. If *sy* matches *sx* the cube will be symetrical on all axis.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the cube. The entire cube becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored
c5	*index*	**index**	This value is ignored
c6	*index*	**index**	This value is ignored

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the pyrimid. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the pyrimid. Good values would be in the 15 to 30 range. If *sy* matches *sx* the pyrimid will be equalateral along it's vertical cross-section.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the pyrimid. The entire pyrimid becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored
c5	*index*	**index**	This value is ignored
c6	*index*	**index**	This value is ignored

Syntax: **G 33** *thickness radius xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Wire Donut**. The geometry type is **33**. The geometry parameters are:

Parameter	Format	Range	Definition
thickness	*float*	**any**	This is the thickness of the donut's ring.
radius	*float*	**any**	This is the radius of the donut.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the donut. The entire donut becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored
c5	*index*	**index**	This value is ignored
c6	*index*	**index**	This value is ignored

Syntax: **G 40** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Solid Sphere**. The geometry type is **40**. This is identical to a type **20** except that level of detail is cranked to maximum quality. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the sphere. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the sphere. Good values would be in the 15 to 30 range. If *sy*

			matches *sx* the sphere will be round.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the sphere. The entire sphere becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 50** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Faceted Sphere**. The geometry type is **50**. This is identical to a type **40** except that the sphere is faceted instead of smooth. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the sphere. Good values would be in the 15 to 30 range.
sy	*float*	**any**	This is the Y size of the sphere. Good values would be in the 15 to 30 range. If *sy* matches *sx* the sphere will be round.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the sphere. The entire sphere becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

Syntax: **G 90** *sx sy xo yo zo xr yr zr c1 c2*

This Geometry command defines a **Solid Teapot**. The geometry type is **90**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the teapot.
sy	*float*	**any**	This is the Y size of the teapot. If *sy* matches *sx* the teapot will be properly shaped.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the teapot. The entire teapot becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored

Syntax:	**G 91** *sx sy xo yo zo xr yr zr c1 c2*

This Geometry command defines a **Wire Teapot**. The geometry type is **91**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the teapot.
sy	*float*	**any**	This is the Y size of the teapot. If *sy* matches *sx* the teapot will be properly shaped.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This is which defined **C** color command to assign to the teapot. The entire teapot becomes this color. The value **0** (zero) means the *first color defined in this Aggregate Group* and so forth...
c2	*index*	**index**	This value is ignored

Syntax:	**G 500** *sx sy xo yo zo xr yr zr c1 c2 c3 c4*

This Geometry command defines a **Floor**. The geometry type is **500**. The geometry parameters are:

Parameter	Format	Range	Definition
sx	*float*	**any**	This is the X size of the floor. Good values would be in the 500 to 1500 range.
sy	*float*	**any**	This is the Y size of the floor. Good values would be in the 500 to 1500 range. If *sy* matches *sx* the floor will be round.
xo	*float*	**any**	This is the X offset from the center for drawing this geometry.
yo	*float*	**any**	This is the Y offset from the center for drawing this geometry.
zo	*float*	**any**	This is the Z offset from the center for drawing this geometry.
xr	*float*	**any**	This is the X rotation angle from the center for drawing this geometry (-360 - +360).
yr	*float*	**any**	This is the Y rotation angle from the center for drawing this geometry (-360 - +360).
zr	*float*	**any**	This is the Z rotation angle from the center for drawing this geometry (-360 - +360).
c1	*index*	**index**	This value is ignored
c2	*index*	**index**	This value is ignored
c3	*index*	**index**	This value is ignored
c4	*index*	**index**	This value is ignored

O - Define an Oscillator

Syntax:	**O 0** *x y z*

This Oscillator command defines an **Add**. The oscillator type is **0** (zero). The oscillator parameters are:

Parameter	Format	Range	Definition
x	*float*	**any**	This value is **added** to the current platforms x position.
y	*float*	**any**	This value is **added** to the current platforms y position.
z	*float*	**any**	This value is **added** to the current platforms z position.

Syntax:	**O 1** *x y z*

This Oscillator command defines a **Scale**. The oscillator type is **1**. The oscillator parameters are:

Parameter	Format	Range	Definition

x	*float*	**any**	This value is **multiplied** with the current platforms x position.
y	*float*	**any**	This value is **multiplied** with the current platforms y position.
z	*float*	**any**	This value is **multiplied** with the current platforms z position.

Syntax: **O 2** *x y z*

This Oscillator command defines an **Index Add**. The oscillator type is **2**. The oscillator parameters are:

Parameter	Format	Range	Definition
x	*float*	**any**	This value is **added** to the current platforms x position and the result is multiplied by the platforms current index.
y	*float*	**any**	This value is **added** to the current platforms y position and the result is multiplied by the platforms current index.
z	*float*	**any**	This value is **added** to the current platforms z position and the result is multiplied by the platforms current index.

Syntax: **O 3** *x y z*

This Oscillator command defines an **Index Scale**. The oscillator type is **3**. The oscillator parameters are:

Parameter	Format	Range	Definition
x	*float*	**any**	This value is **multiplied** with the current platforms x position and the result is multiplied by the platforms current index.
y	*float*	**any**	This value is **multiplied** with the current platforms y position and the result is multiplied by the platforms current index.
z	*float*	**any**	This value is **multiplied** with the current platforms z position and the result is multiplied by the platforms current index.

Syntax: **O 4** *x y z*

This Oscillator command defines an **Index Scale with Zero skip**. The oscillator type is **4**. The oscillator parameters are:

Parameter	Format	Range	Definition
x	*float*	**any**	If this axis is non-zero than this value is **multiplied** with the current platforms x position and the result is multiplied by the platforms current index.
y	*float*	**any**	If this axis is non-zero than this value is **multiplied** with the current platforms y position and the result is multiplied by the platforms current index.
z	*float*	**any**	If this axis is non-zero than this value is **multiplied** with the current platforms z position and the result is multiplied by the platforms current index.

Syntax: **O 50** *frequency phase sweep amplitude*

This Oscillator command defines a **Sawtooth X** oscillator. The oscillator type is **50**. The oscillator parameters are:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sawtooth wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
phase	*float*	**any**	This parameter is similar to *amplitude* except this paramter is multiplied before the mod function whereas *amplitude* is multiplied after the mod function. Setting this value to **1.0** will remove it from the picture.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal

			values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 51** *frequency phase sweep amplitude*

This Oscillator command defines a **Sawtooth Y** oscillator. The oscillator type is **51**. The oscillator parameters are:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sawtooth wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
phase	*float*	**any**	This parameter is similar to *amplitude* except this paramter is multiplied before the mod function whereas *amplitude* is multiplied after the mod function. Setting this value to **1.0** will remove it from the picture.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 52** *frequency phase sweep amplitude*

This Oscillator command defines a **Sawtooth *Z*** oscillator. The oscillator type is **52**. The oscillator parameters are:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sawtooth wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
phase	*float*	**any**	This parameter is similar to *amplitude* except this paramter is multiplied before the mod function whereas *amplitude* is multiplied after the mod function. Setting this value to **1.0** will remove it from the picture.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 100** *frequency sweep amplitude*

This Oscillator command defines an **X * SIN**. The oscillator type is **100**. In this case the platform's X coordinate is multiplied times a SIN value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 101** *frequency sweep amplitude*

This Oscillator command defines an **Y * SIN**. The oscillator type is **101**. In this case the platform's Y coordinate is multiplied times a SIN value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 102** *frequency sweep amplitude*

This Oscillator command defines an **Z * SIN**. The oscillator type is **102**. In this case the platform's Z coordinate is multiplied times a SIN value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the

			more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax:	**O 110** *frequency sweep amplitude*

This Oscillator command defines an **X * COS**. The oscillator type is **110**. In this case the platform's X coordinate is multiplied times a COS value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax:	**O 111** *frequency sweep amplitude*

This Oscillator command defines an **Y * COS**. The oscillator type is **111**. In this case the platform's Y coordinate is multiplied times a COS value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax:	**O 112** *frequency sweep amplitude*

This Oscillator command defines an **Z * COS**. The oscillator type is **112**. In this case the platform's Z coordinate is multiplied times a COS value which is generated from the following parameters:

Parameter	Format	Range	Definition
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting sine wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative

			counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
sweep	*float*	**any**	This value specifies how each platform reacts to the *frequency*. This value is multplied times the index of the platform as well as the frequency, thus this value determines the number of waves will appear throughout the length. A value of zero will turn wave off, thus if *frequency* is non-zero and this value is zero you will produce a straight line where all platforms move together. The larger this value becomes the more each platform is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 200** *oscillator argument offset frequency amplitude*

This Oscillator command defines a **CROSSPATCH O-TYPE SIN LOW FREQUENCY** oscillator. The oscillator type is **200**. Crosspatch O oscillators allow you modulate another oscillator with this oscillator. This is useful for producing very complex patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
oscillator	*long*	**0 - 31**	This parameter specifies which oscillator you wish to modulate. The oscillator must be defined in the current Aggregate Group. Since the first oscillator starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the oscillator specified by *oscillator* to modulate. The oscillator type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the oscillator type id. Whatever that parameter is will be multiplied by the sin of the value that results from this operation.
offset	*float*	**-360 - +360**	This parameter specifies an offset to the phase of the LFO. An offset of **0** (zero) produces no offset. An offset of **180** for example would produce a wave that is 180 degrees out of phase with a wave produced with an offset of zero.
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 210** *oscillator argument offset frequency amplitude*

This Oscillator command defines a **CROSSPATCH O-TYPE COS LOW FREQUENCY** oscillator. The oscillator type is **210**. Crosspatch O oscillators allow you modulate another oscillator with this oscillator. This is useful for producing very complex patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
oscillator	*long*	**0 - 31**	This parameter specifies which oscillator you wish to modulate. The oscillator must be defined in the current Aggregate Group. Since the first oscillator starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the oscillator specified by *oscillator* to modulate. The oscillator type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the oscillator type id. Whatever that parameter is will be multiplied by the cos of the value that results from this operation.
offset	*float*	**-360 - +360**	This parameter specifies an offset to the phase of the LFO. An offset of **0** (zero) produces no offset. An offset of **180** for example would produce a wave that is 180 degrees out of phase with a wave produced with an offset of zero.
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion

			to the resulting wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 220** *oscillator argument increment*

This Oscillator command defines a **CROSSPATCH O-TYPE INCREMENT** oscillator. The oscillator type is **220**. Crosspatch O oscillators allow you to modulate an oscillator using this oscillator. This is useful in producing very complex platform movement patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
oscillator	*long*	**0 - n**	This parameter specifies which oscillator command you wish to modulate. The oscillator command must be defined in the current Aggregate Group. Since the first oscillator command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id.
increment	*float*	**any**	This parameter specifes the increment to be added to the *color* commands *argument.*

Syntax: **O 230** *oscillator argument scale*

This Oscillator command defines a **CROSSPATCH O-TYPE SCALE** oscillator. The oscillator type is **230**. Crosspatch O oscillators allow you to modulate an oscillator using this oscillator. This is useful in producing very complex platform movement patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
oscillator	*long*	**0 - n**	This parameter specifies which oscillator command you wish to modulate. The oscillator command must be defined in the current Aggregate Group. Since the first oscillator command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the oscillator command specified by *oscillator* to modulate. The oscillator type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the oscillator type id.
scale	*float*	**any**	This parameter specifes the scale to be multiplied with the *oscillator* commands *argument.*

Syntax: **O 240** *oscillator argument amplitude*

This Oscillator command defines a **CROSSPATCH O-TYPE PROXIMITY** oscillator. The oscillator type is **240**. Crosspatch O oscillators allow you to modulate an oscillator using this oscillator. This is useful in producing very complex platform movement patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
oscillator	*long*	**0 - n**	This parameter specifies which oscillator command you wish to modulate. The oscillator command must be defined in the current Aggregate Group. Since the first oscillator command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the oscillator command specified by *oscillator* to modulate. The oscillator type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the oscillator type id.
amplitude	*float*	**any**	This parameter specifes the amount to multiply the value based on the users proximity to the object.

Syntax: **O 300** *color argument offset frequency amplitude*

This Oscillator command defines a **CROSSPATCH C-TYPE SIN LOW FREQUENCY** oscillator. The oscillator type is **300**. Crosspatch C oscillators allow you modulate a color command with this oscillator. This is useful for producing very complex color patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
color	*long*	**0 - n**	This parameter specifies which color command you wish to modulate. The color command must be defined in the current Aggregate Group. Since the first color command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id. Whatever that parameter is will be multiplied by the sin of the value that results from this operation.
offset	*float*	**-360 - +360**	This parameter specifies an offset to the phase of the LFO. An offset of **0** (zero) produces no offset. An offset of **180** for example would produce a wave that is 180 degrees out of phase with a wave produced with an offset of zero.
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 310** *color argument offset frequency amplitude*

This Oscillator command defines a **CROSSPATCH C-TYPE COS LOW FREQUENCY** oscillator. The oscillator type is **310**. Crosspatch C oscillators allow you modulate a color command with this oscillator. This is useful for producing very complex color patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
color	*long*	**0 - n**	This parameter specifies which color command you wish to modulate. The color command must be defined in the current Aggregate Group. Since the first color command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id. Whatever that parameter is will be multiplied by the cos of the value that results from this operation.
offset	*float*	**-360 - +360**	This parameter specifies an offset to the phase of the LFO. An offset of **0** (zero) produces no offset. An offset of **180** for example would produce a wave that is 180 degrees out of phase with a wave produced with an offset of zero.
frequency	*float*	**any**	This value can be thought of as specifying an amount of wavyness or motion to the resulting wave. If this value is zero motion is muted and the platforms stay at their assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion (wave) into the final structure. This value is a multiplier to the clock so small values produce more gentle motions while large values produce more extreme motions.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces a straight line since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 320** *color argument increment*

This Oscillator command defines a **CROSSPATCH C-TYPE INCREMENT** oscillator. The oscillator type is **320**. Crosspatch C oscillators allow you modulate a color command with this oscillator. This is useful for producing very complex color patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
color	*long*	**0 - n**	This parameter specifies which color command you wish to modulate. The color command must be defined in the current Aggregate Group. Since the first color command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id.
increment	*float*	**any**	This parameter specifes the increment to be added to the *color* commands *argument.*

Syntax: **O 330** *color argument scale*

This Oscillator command defines a **CROSSPATCH C-TYPE SCALE** oscillator. The oscillator type is **330**. Crosspatch C oscillators allow you modulate a color command with this oscillator. This is useful for producing very complex color patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
color	*long*	**0 - n**	This parameter specifies which color command you wish to modulate. The color command must be defined in the current Aggregate Group. Since the first color command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id.
scale	*float*	**any**	This parameter specifes the scale to be multiplied with the *color* commands *argument.*

Syntax: **O 340** *color argument amplitude*

This Oscillator command defines a **CROSSPATCH C-TYPE PROXIMITY** oscillator. The oscillator type is **340**. Crosspatch C oscillators allow you modulate a color command with this oscillator. This is useful for producing very complex color patterns. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
color	*long*	**0 - n**	This parameter specifies which color command you wish to modulate. The color command must be defined in the current Aggregate Group. Since the first color command starts at **0** (zero) all numbering begins there.
argument	*long*	**varies**	This pararmeter specifies which argument of the color command specified by *color* to modulate. The color type id (first parameter) is ignored so an *argument* of **0** (zero) indicates to start with the first parameter following the color type id.
amplitude	*float*	**any**	This parameter specifes the amount to multiply the value based on the users proximity to the object.

Syntax: **O 350** *geometry rxi ryi rzi*

This Oscillator command defines a **ADD G-TYPE ROTATE** oscillator. The oscillator type is **350**. In this case the parameters specified are added to the indicated geometry current rotation values. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the rotation of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
rxi	*float*	**varies**	The amount to add to the indicated geometries X rotation value.
ryi	*float*	**varies**	The amount to add to the indicated geometries Y rotation value.
rzi	*float*	**varies**	The amount to add to the indicated geometries Z rotation value.

Syntax: **O 351** *geometry rxi ryi rzi*

This Oscillator command defines a **G-TYPE PROXIMITY ROTATE** oscillator. The oscillator type is **351**. In this case the specified geometries rotation elements are set to the specified values based on the users distance from the object.. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the rotation of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
rxi	*float*	**varies**	The maximum amount to set the indicated geometries X rotation value. This is multiplied times the users distance from the object and the objects X rotation is set to this value.
ryi	*float*	**varies**	The maximum amount to set the indicated geometries Y rotation value. This is multiplied times the users distance from the object and the objects Y rotation is set to this value.
rzi	*float*	**varies**	The maximum amount to set the indicated geometries Z rotation value. This is multiplied times the users distance from the object and the objects Z rotation is set to this value.

Syntax: **O 352** *geometry rxi ryi rzi*

This Oscillator command defines a **ADD G-TYPE PROXIMITY ROTATE** oscillator. The oscillator type is **352**. In this case the specified geometries rotation elements are added to the specified values based on the users distance from the object.. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the rotation of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
rxi	*float*	**varies**	The maximum amount to add to the indicated geometries X rotation value. This value is multiplied times the users distance from the object and the objects X rotation is added to the result.
ryi	*float*	**varies**	The maximum amount to add to the indicated geometries Y rotation value. This value is multiplied times the users distance from the object and the objects Y rotation is added to the result.
rzi	*float*	**varies**	The maximum amount to add to the indicated geometries Z rotation value. This value is multiplied times the users distance from the object and the objects Z rotation is added to the result.

Syntax: **O 360** *geometry freqency sweep amplitude*

This Oscillator command defines a **G-TYPE X SIN** oscillator. The oscillator type is **360**. In this case the specified geometries X offset elements are set to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 361** *geometry freqency sweep amplitude*

This Oscillator command defines a **G-TYPE Y SIN** oscillator. The oscillator type is **361**. In this case the specified geometries Y offset elements are set to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 362** *geometry freqency sweep amplitude*

This Oscillator command defines a **G-TYPE Z SIN** oscillator. The oscillator type is **362**. In this case the specified geometries Z offset elements are set to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 363** *geometry freqency sweep amplitude*

This Oscillator command defines a **ADD G-TYPE X SIN** oscillator. The oscillator type is **363**. In this case the specified geometries X offset elements are added to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is

			multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 364** *geometry freqency sweep amplitude*

This Oscillator command defines a **ADD G-TYPE Y SIN** oscillator. The oscillator type is **364**. In this case the specified geometries Y offset elements are added to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 365** *geometry freqency sweep amplitude*

This Oscillator command defines a **ADD G-TYPE Z SIN** oscillator. The oscillator type is **362**. In this case the specified geometries Z offset elements are added to the SIN of the values specified in this command. This oscillator takes the following parameters:

Parameter	Format	Range	Definition
geometry	*long*	**0 - 31**	This parameter specifies which geometry command you wish to modify the offset of. The geometry command must be defined in the current Aggregate Group. Since the first geometry command starts at **0** (zero) all numbering begins there.
frequency	*float*	**any**	This is the frequency (amount of motion) to impart to the offset. If this value is zero motion is muted and the offset stays at its assigned position. Very small values (in the range of .1 to .005 or their negative counterparts) can be made to impart motion into the offset.
sweep	*float*	**any**	This value specifies how each geometry reacts to the *frequency*. This value is multplied times the index of the geometry as well as the frequency, thus this value determines the amount of rotations that will appear throughout the length. A value of zero will turn rotation off, thus if *frequency* is non-zero and this value is zero all geometry will rotate the same together. The larger this value becomes the more each geometry is offset from the previous based on the *frequency*. Normal values for this are quite large, 100 to 500 work well.
amplitude	*float*	**any**	This value specifies how exagerated the peaks of the waveforms are. A value of zero produces no rotation since it mutes the peaks. Higher values produce waves with larger peaks. Good values range from 1 on up to thousands depending on the effect your trying to obtain.

Syntax: **O 900** *xpull ypull zpull*

This Oscillator command defines a **FLOCK** oscillator. The oscillator type is **900**. In this case platforms will either flock towards or away from the player. Small platforms can appear as little gnats that follow and frustrate the player since they can be set to collide if desired. Parameters for this oscillator are:

Parameter	Format	Range	Definition
xpull	*float*	**any**	This controls the speed at which the platforms follow the player in the x axis. A value of zero causes platforms to not follow in this axis. A positive value will cause platforms to move TOWARDS the user in this axis while a negative value will cause platforms to move AWAY from the user in this axis. The higher the value the faster the movement.
ypull	*float*	**any**	This controls the speed at which the platforms follow the player in the y axis. A value of zero causes platforms to not follow in this axis. A positive value will cause platforms to move TOWARDS the user in this axis while a negative value will cause platforms to move AWAY from the user in this axis. The higher the value the faster the movement.
zpull	*float*	**any**	This controls the speed at which the platforms follow the player in the z axis. A value of zero causes platforms to not follow in this axis. A positive value will cause platforms to move TOWARDS the user in this axis while a negative value will cause platforms to move AWAY from the user in this axis. The higher the value the faster the movement.

Syntax: **O 910** *xamp yamp zamp*

This Oscillator command defines a **BROWNIAN** oscillator. The oscillator type is **910**. In this case a brownian motion is imparted to a platforms axis based on an amplitude multiply. Brownian motion is a noise motion and thus imparts a jitter to a platform:

Parameter	Format	Range	Definition
xamp	*float*	**any**	This is the amplitude of the brownian noise you wish multplied times the platforms x axis. A value of zero turns off noise for this axis. The higher this value the more platforms jitter. A value of 100 is a fairly nasty jitter. Note that platforms will tend to stray appart in this axis if it is non-zero.
yamp	*float*	**any**	This is the amplitude of the brownian noise you wish multplied times the platforms y axis. A value of zero turns off noise for this axis. The higher this value the more platforms jitter. A value of 100 is a fairly nasty jitter. Note that platforms will tend to stray appart in this axis if it is non-zero.
zamp	*float*	**any**	This is the amplitude of the brownian noise you wish multplied times the platforms z axis. A value of zero turns off noise for this axis. The higher this value the more platforms jitter. A value of 100 is a fairly nasty jitter. Note that platforms will tend to stray appart in this axis if it is non-zero.

P - Define a platform

Syntax: **P** *oscbits count reveal geomup geomdn eventsup eventsdn pingup pingdn initx inity initz sizex sizey sizez stepy stepz hitbits lstart lend*

The **P** command implements a platform. **P**'s are required in order to actually display geometry associated with oscilators. Platforms have the following parameters:

Parameter	Format	Range	Definition
oscbits	*string*	**0's and 1's**	This sets the oscillator bits used by this platform. These are executed in MSB to LSB order, thus the first character specifies the first oscillator defined.
count	*long*	**1-9000**	This sets the total number of platforms to allocate for this particular platform statement.
reveal	*long*	**1-*count***	This sets the total number of platforms to show for this particular platform statement. This number must be the same as *count* or less than it. No more than *reveal* platforms will be visible for this **P** statement.
geomup	*long*	**bit index**	This is a long where the bits define which **G** (geometry) from the current aggregrate are affected by this platform command. This sets the geometry of the underside of the object (that is, the side seen when looking up).
geomdn	*long*	**bit index**	This is a long where the bits define which **G** (geometry) from the current aggregrate are affected by this platform command. This sets the geometry of the topside of the object (that is, the side seen when looking down).
eventsup	*string*	**0's and 1's**	This sets which events are triggered when the user collides with the platform from below.

eventsdn	*string*	**0's and 1's**	This sets which events are triggered when the user collides with the platform from above.
pingup	*float*	**any**	This is a multiplier used to factor the amount of lift the user gets when they hit this platform from below.
pingdn	*float*	**any**	This is a multiplier used to factor the amount of lift the user gets when they hit this platform from above.
initx	*float*	**any**	This is the starting X offset of where this platform is drawn before oscillators are applied.
inity	*float*	**any**	This is the starting Y offset of where this platform is drawn before oscillators are applied.
initz	*float*	**any**	This is the starting Z offset of where this platform is drawn before oscillators are applied.
sizex	*float*	**any**	This is the size in the X axis of the collision rectangle for this platform.
sizey	*float*	**any**	This is the size in the Y axis of the collision rectangle for this platform.
sizez	*float*	**any**	This is the size in the Z axis of the collision rectangle for this platform.
stepx	*float*	**any**	This is the amount to add to the X position for each successive platform (for *reveal* platforms) as it is created.
stepy	*float*	**any**	This is the amount to add to the Y position for each successive platform (for *reveal* platforms) as it is created.
stepz	*float*	**any**	This is the amount to add to the Z position for each successive platform (for *reveal* platforms) as it is created.
hitbits	*long*	**0 or 1**	If this is a **1** than the platform disappears upon colission.
lstart	*float*	**any**	This is the room level start for this platform. If the current room level is below *lstart* than these platforms are not displayed.
lend	*float*	**any**	This is the room level end for this platform. If the current room level is above *lend* than these platforms are not displayed.

© 1992-2003 TQworld, LLC